

Using the STAR-System Triggering API

STAR-Dundee's USB Brick Mk3 and PXI SpaceWire Interface boards provide triggering capabilities to allow events occurring on the devices to cause actions such as pulsing external triggers, transmitting time-codes, transmitting packets or injecting errors. In addition to the physical triggering resources such as external triggers and SpaceWire ports, a set of internal counters and triggers allow a user application to set up advanced triggering scenarios.

The API provided with STAR-System provides functions to configure the triggering resources found on the boards, filter triggering events into internal triggers and set up the internal triggers to cause triggering actions.

This application note first explains the concepts of events, internal triggers and actions, then provides examples showing how the Triggering API can be used to inject errors after receiving an external trigger pulse, transmit packets periodically and use an internal counter to transmit groups of packets whenever a time-code is received. Further information on the API and the functions provided can be found in the STAR-System documentation.

Events, Internal Triggers and Actions

The Triggering API is based around three concepts: events, internal triggers and actions.

Events occur on external triggers, internal counters, SpaceWire ports, time-code interfaces and internal triggers. For example:

- [EXT_TRIGGER_EVENT_IN](#) events occur when an external trigger ports receives a pulse
- [COUNTER_EVENT_COUNT](#) events occur when a counter is decremented
- [PORT_EVENT_RX_EEP](#) events occur when a SpaceWire port receives an error-end-of-packet
- [TIME_CODE_EVENT_TICK](#) events occur when the device's time-code value is incremented

These events can cause one or more internal triggers to be set, which can then be used to cause different actions on the external triggers, internal counters, SpaceWire ports and time-code interfaces. For example:

- [EXT_TRIGGER_ACTION_OUT](#) actions cause an external trigger to output a pulse
- [COUNTER_ACTION_RELOAD](#) actions cause a counter to be reloaded
- [PORT_ACTION_TRANSMIT_PKT](#) actions cause a SpaceWire port to transmit a packet
- [TIME_CODE_ACTION_TX](#) actions cause the next time-code to be transmitted

The Triggering API provides the following functions to control which input events cause internal triggers to be set (the wildcard in the function name is replaced by the type of device e.g. BRICK_MK3 or PXI_IF):

- [TRIGGER_*_setExtTriggerInputEvents\(\)](#),
[TRIGGER_*_getExtTriggerInputEvents\(\)](#)

These functions can be used to set and get the external trigger events that cause an internal trigger to be set. In this case, there is only one available event which is the [EXT_TRIGGER_EVENT_IN](#) event which occurs whenever the external trigger receives a pulse. The set function takes a device identifier, an external trigger index, an internal trigger index and a mask describing the external trigger events that will cause the internal trigger to be set. The get

function takes a device identifier, an external trigger index, an internal trigger index and a pointer to a value that will be updated with a mask describing the external trigger events that will cause the internal trigger to be set.

- [TRIGGER_*_setCounterInputEvents\(\)](#),
[TRIGGER_*_getCounterInputEvents\(\)](#)

These functions can be used to set and get the counter events that cause an internal trigger to be set. For example, the [COUNTER_EVENT_COUNT](#) event occurs when the counter is decremented, the [COUNTER_EVENT_ZERO_SINGLE](#) event occurs when the counter hits zero and the [COUNTER_EVENT_RELOAD](#) event occurs when the counter is reloaded. The set function takes a device identifier, a counter index, an internal trigger index and a mask describing the counter events that will cause the internal trigger to be set. The get function takes a device identifier, a counter index, an internal trigger index and a pointer to a value that will be updated with a mask describing the counter events that will cause the internal trigger to be set.

- [TRIGGER_*_setPortInputEvents\(\)](#),
[TRIGGER_*_getPortInputEvents\(\)](#)

These functions can be used to set and get the SpaceWire port events that cause an internal trigger to be set. For example, the [PORT_EVENT_RX_SOP](#) occurs when the port receives a start-of-packet, the [PORT_EVENT_PARITY_ERROR](#) event occurs when the port detects a parity error and the [PORT_EVENT_DISCONNECT](#) event occurs when the port is disconnected. The other event types are described in the STAR-System documentation. The set function takes a device identifier, a port number, an internal trigger index and a mask describing the port events that will cause the internal trigger to be set. The get function takes a device identifier, a port number, an internal trigger index and a pointer to a value that will be updated with a mask describing the port events that will cause the internal trigger to be set.

- [TRIGGER_*_setTimeCodeInputEvents\(\)](#),
[TRIGGER_*_getTimeCodeInputEvents\(\)](#)

These functions can be used to set and get the time-code interface events that cause an internal trigger to be set. In this case, there is only one available event which is the [TIME_CODE_EVENT_TICK](#) event which occurs whenever the device's internal time-code value is incremented. The set function takes a device identifier, a time-code interface index, an internal trigger index and a mask describing the time-code interface events that will cause the internal trigger to be set. The get function takes a device identifier, a time-code interface index, an internal trigger index and a pointer to a value that will be updated with a mask describing the time-code interface events that will cause the internal trigger to be set.

- [TRIGGER_*_setTriggerInputEvents\(\)](#),
[TRIGGER_*_getTriggerInputEvents\(\)](#)

These functions can be used to set and get the internal trigger events that cause another internal trigger to be set. In this case, there is only one available event which is the [TRIGGER_EVENT_IN](#) event which occurs whenever the internal trigger is set. The set function takes a device identifier, an internal trigger index, another internal trigger index and a mask describing the internal trigger events that will cause the other internal trigger to be set. The get function takes a device identifier, an internal trigger index, another internal trigger index and a pointer to a value that will be updated with a mask describing the internal trigger events that will cause the other internal trigger to be set.

The Triggering API provides the following functions to control which output actions are caused by internal triggers being set (the wildcard in the function name is replaced by the type of device e.g. BRICK_MK3 or PXI_IF):

- [TRIGGER_*_setExtTriggerOutputActions\(\)](#),
[TRIGGER_*_getExtTriggerOutputActions\(\)](#)

These functions can be used to set and get the external trigger actions that are caused by an



internal trigger being set. In this case, there is only one available action which is the [EXT_TRIGGER_ACTION_OUT](#) action which pulses an external trigger for a configured duration. The set function takes a device identifier, an external trigger index, an internal trigger index and a mask describing the external trigger actions that will be caused by the internal trigger being set. The get function takes a device identifier, an external trigger index, an internal trigger index and a pointer to a value that will be updated with a mask describing the external trigger actions that will be caused by the internal trigger being set.

- [TRIGGER_*_setCounterOutputActions\(\)](#),
[TRIGGER_*_getCounterOutputActions\(\)](#)

These functions can be used to set and get the counter actions that are caused by an internal trigger being set. For example, the [COUNTER_ACTION_COUNT](#) action causes the counter to be decremented, the [COUNTER_ACTION_RELOAD](#) event causes the counter to be reloaded and the [COUNTER_ACTION_START](#) and [COUNTER_ACTION_STOP](#) actions cause the counter to start and stop. The other action types are described in the STAR-System documentation. The set function takes a device identifier, a counter index, an internal trigger index and a mask describing the counter actions that will be caused by the internal trigger being set. The get function takes a device identifier, a counter index, an internal trigger index and a pointer to a value that will be updated with a mask describing the counter actions that will be caused by the internal trigger being set.

- [TRIGGER_*_setPortOutputActions\(\)](#),
[TRIGGER_*_getPortOutputActions\(\)](#)

These functions can be used to set and get the SpaceWire port actions that are caused by an internal trigger being set. For example, the [PORT_ACTION_TRANSMIT_PKT](#) action causes a queued packet to be transmitted on the port if it has been configured to queue and transmit packets when triggered, the [PORT_ACTION_DISCONNECT](#) action causes the port to be disconnected and the [PORT_ACTION_PARITY_ERROR](#) action causes a parity error to be injected on the port. The other action types are described in the STAR-System documentation. The set function takes a device identifier, a port number, an internal trigger index and a mask describing the port actions that will be caused by the internal trigger being set. The get function takes a device identifier, a port number, an internal trigger index and a pointer to a value that will be updated with a mask describing the port actions that will be caused by the internal trigger being set.

- [TRIGGER_*_setTimeCodeOutputActions\(\)](#),
[TRIGGER_*_getTimeCodeOutputActions\(\)](#)

These functions can be used to set and get the time-code interface actions that are caused by an internal trigger being set. In this case, there is only one available action which is the [TIME_CODE_ACTION_TX](#) action which causes the next time-code to be transmitted. The set function takes a device identifier, a time-code interface index, an internal trigger index and a mask describing the time-code interface actions that will be caused by the internal trigger being set. The get function takes a device identifier, a time-code interface index, an internal trigger index and a pointer to a value that will be updated with a mask describing the time-code interface actions that will be caused by the internal trigger being set.

In addition to the functions used to control the input events and output actions, there are a set of configuration functions. For example, there are functions to enable an external trigger's edge detect mode and set its output duration; set an internal counter to automatic or triggered counting mode, allowing the counter to be used as a timer or counter; or enable a SpaceWire port's packet transmit mode, allowing it to send packets triggered by a timer or other event. The full list of configuration functions is provided in the STAR-System documentation.



Injecting Errors Using an External Trigger

The example below shows how to use the Triggering API for a USB Brick Mk3 to inject a parity error on port 1 whenever external trigger port 0 receives a pulse.

Note that the example assumes the application already has the device ID of a USB Brick Mk3 device which is stored in the "deviceId" variable.

```
int edgeSuccess;
int eventSuccess;
int actionSuccess;
int enableSuccess;

/* Enable edge detect mode on external trigger 0 so that the */
/* EXT_TRIGGER_EVENT_IN event occurs once on the rising edge of a */
/* pulse received by external trigger 0 */
edgeSuccess = TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode(deviceId, 0);
if(!edgeSuccess)
{
    printf("Enabling edge detect mode failed.\n", i);
    return;
}

/* Set up the EXT_TRIGGER_EVENT_IN event on external trigger 0 to cause */
/* internal trigger 0 to be set */
eventSuccess = TRIGGER_BRICK_MK3_setExtTriggerInputEvents(deviceId, 0, 0,
    EXT_TRIGGER_EVENT_IN);
if(!eventSuccess)
{
    printf("Setting the external trigger input events failed.\n", i);
    return;
}

/* Set up the PORT_ACTION_PARITY_ERROR action on port 1 to be caused */
/* by internal trigger 0 being set */
actionSuccess = TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 0,
    PORT_ACTION_PARITY_ERROR);
if(!actionSuccess)
{
    printf("Setting the port output actions failed.\n", i);
    return;
}

/* Enable internal trigger 0 */
enableSuccess = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
if(!enableSuccess)
{
    printf("Enabling the internal trigger failed.\n", i);
    return;
}
```

Transmitting Packets Periodically

The example below shows how to use the Triggering API for a USB Brick Mk3 to transmit packets on port 1 every 100000 clock cycles using a counter.



Note, for brevity, error checking for the API calls is not included in this example.

```
/* Set the reload value of counter 0 to 100000 clock cycles */
TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 0, 100000);

/* Enable auto reload for counter 0 so it is reloaded whenever it expires */
TRIGGER_BRICK_MK3_enableCounterAutoReload(deviceId, 0);

/* Disable trigger count for counter 0 so it is decremented automatically */
/* every clock cycle */
TRIGGER_BRICK_MK3_disableCounterTriggerCount(deviceId, 0);

/* Enable transmit packet mode for port 1 so that any packets sent on the */
/* port are queued until they are triggered */
TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(deviceId, 1);

/* Set up the COUNTER_EVENT_RELOAD event on counter 0 to cause */
/* internal trigger 0 to be set */
TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 0, 0, COUNTER_EVENT_RELOAD);

/* Set up the PORT_ACTION_TRANSMIT_PKT action on port 1 to be caused */
/* by internal trigger 0 being set */
TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 0, PORT_ACTION_TRANSMIT_PKT);

/* Enable internal trigger 0 */
TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
```

Transmitting Groups of Packets After Receiving Time-Codes

The example below shows how to use the Triggering API for a USB Brick Mk3 to transmit groups of 4 packets on port 1 every time the device receives a valid time-code. The first packet of each group is transmitted when the time-code is received. The remaining three are transmitted using a counter which counts down every time an EOP is transmitted from 3 to 0, which then causes the next packet to be transmitted.

Note, for brevity, error checking for the API calls is not included in this example.

```
/* Enable trigger count mode for trigger 0 so that it is only */
/* decremented when triggered */
TRIGGER_BRICK_MK3_enableCounterTriggerCount(deviceId, 0);

/* Disable auto reload for counter 0 */
TRIGGER_BRICK_MK3_disableCounterAutoReload(deviceId, 0);

/* Set the reload value of counter 0 to 3 so that it can be used to */
/* count how many packets are transmitted after the first */
TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 0, 3);

/* Enable transmit packet mode for port 1 so that any packets sent on the */
/* port are queued until they are triggered */
TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(deviceId, 1);

/* Set up the TIME_CODE_EVENT_TICK event on time-code interface 0 to cause */
/* internal trigger 0 to be set */
TRIGGER_BRICK_MK3_setTimeCodeInputEvents(deviceId, 0, 0, TIME_CODE_EVENT_TICK);
```



```

/* Set up the COUNTER_ACTION_RELOAD action on counter 0 to be caused */
/* by internal trigger 0 being set */
TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 0, 0, COUNTER_ACTION_RELOAD);

/* Set up the PORT_ACTION_TRANSMIT_PKT action on port 1 to be caused */
/* by internal trigger 0 being set */
TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 0, PORT_ACTION_TRANSMIT_PKT);

/* Set up the PORT_EVENT_TX_EOP event on port 1 to cause */
/* internal trigger 1 to be set */
TRIGGER_BRICK_MK3_setPortInputEvents(deviceId, 1, 1, PORT_EVENT_TX_EOP);

/* Set up the COUNTER_ACTION_COUNT action on counter 0 to be caused */
/* by internal trigger 1 being set */
TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 0, 1, COUNTER_ACTION_COUNT);

/* Set up the COUNTER_EVENT_COUNT event on counter 0 to cause */
/* internal trigger 2 to be set */
TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 0, 2, COUNTER_EVENT_COUNT);

/* Set up the PORT_ACTION_TRANSMIT_PKT action on port 1 to be caused */
/* by internal trigger 2 being set */
TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 2, PORT_ACTION_TRANSMIT_PKT);

/* Enable internal triggers 0, 1 and 2 */
TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
TRIGGER_BRICK_MK3_enableTrigger(deviceId, 1);
TRIGGER_BRICK_MK3_enableTrigger(deviceId, 2);

```

